HDOC Day-57

Maximum Contiguous Subarray Sum Using Kadane's Algorithm in C

Problem Statement

Read the size and elements of a variable-length integer array. Pass the array address to max_subarray_sum() using pointers. Apply Kadane's algorithm to find and display the maximum sum of a non-empty contiguous subarray. For an all-negative array, display its largest (least negative) element.

Algorithm

1. Start.

2. Read array size n. If n is not a positive integer (or exceeds the practical limit), display an error and stop.

3. Declare the variable-length integer array arr[n].

4. Read n integers into arr.

5. Pass arr and n to max_subarray_sum(arr, n) using a pointer.

6. Set current = arr[0] and best = arr[0], so a non-empty subarray is always considered.

7. For each element arr[i] from i = 1 to n-1, set current to the larger of arr[i] and current + arr[i].

8. If current > best, update best = current.

9. Print best as the maximum subarray sum.

10. Stop.

Test Case Table

TC	Size and input array	Expected output	Test type	Input instructions
1	n=9; -2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6	Professor example: mixed numbers	Enter n, then the array elements
2	n=6; -8 -3 -6 -2 -5 -4	Maximum subarray sum: -2	Professor example: all negative	Enter n, then the array elements
3	n=4; 5 -2 3 4	Maximum subarray sum: 10	Professor example: whole array	Enter n, then the array elements
4	n=5; 1 2 3 4 5	Maximum subarray sum: 15	All positive	Enter n, then the array elements
5	n=4; -5 0 -2 -8	Maximum subarray sum: 0	Contains zero and negatives	Enter n, then the array elements
6	n=1; 7	Maximum subarray sum: 7	Single positive element	Enter n, then the array elements
7	n=1; -7	Maximum subarray sum: -7	Single negative element	Enter n, then the array elements
8	n=5; -2 -3 -1 -4 -6	Maximum subarray sum: -1	All negative with different maximum	Enter n, then the array elements
9	n=5; 2 -1 2 -1 2	Maximum subarray sum: 4	Alternating signs	Enter n, then the array elements
10	n=0; (no elements)	Invalid size. Enter an integer from 1 to 10000.	Invalid zero array size	Enter 0 only

C Program

```
#include <stdio.h>

void max_subarray_sum(const int *arr, int size)
{
    long long current = *arr;
    long long best = *arr;
    int i;

    for (i = 1; i < size; i++)
    {
        long long value = *(arr + i);
        if (current + value > value)
            current = current + value;
        else
            current = value;

        if (current > best)
            best = current;
    }

    printf("Maximum subarray sum: %lld\n", best);
}

int main(void)
{
    int size, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000)
    {
        printf("Invalid size. Enter an integer from 1 to 10000.\n");
        return 1;
    }

    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid array element.\n");
            return 1;
        }
    }

    max_subarray_sum(arr, size);
    return 0;
}
```

Compilation and Execution

```
gcc hdoc57.c -o hdoc57
./hdoc57
```

Screenshot Spaces

Compilation and Execution

Show the gcc compilation command, the ./hdoc57 execution command, and a successful program run.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc57.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc57.c -o hdoc57
```

Test Case 1

Input: size = 9; array = -2 1 -3 4 -1 2 1 -5 4. Expected: Maximum subarray sum: 6. (Professor example: mixed numbers)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 9
Enter 9 array elements:
-2 1 -3 4 -1 2 1 -5 4
Maximum subarray sum: 6
```

Test Case 2

Input: size = 6; array = -8 -3 -6 -2 -5 -4. Expected: Maximum subarray sum: -2. (Professor example: all negative)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 6
Enter 6 array elements:
-8 -3 -6 -2 -5 -4
Maximum subarray sum: -2
```

Test Case 3

Input: size = 4; array = 5 -2 3 4. Expected: Maximum subarray sum: 10. (Professor example: whole array)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 4
Enter 4 array elements:
5 -2 3 4
Maximum subarray sum: 10
```

Test Case 4

Input: size = 5; array = 1 2 3 4 5. Expected: Maximum subarray sum: 15. (All positive)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 5
Enter 5 array elements:
1 2 3 4 5
Maximum subarray sum: 15
```

Test Case 5

Input: size = 4; array = -5 0 -2 -8. Expected: Maximum subarray sum: 0. (Contains zero and negatives)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 4
Enter 4 array elements:
-5 0 -2 -8
Maximum subarray sum: 0
```

Test Case 6

Input: size = 1; array = 7. Expected: Maximum subarray sum: 7. (Single positive element)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 1
Enter 1 array elements:
7
Maximum subarray sum: 7
```

Test Case 7

Input: size = 1; array = -7. Expected: Maximum subarray sum: -7. (Single negative element)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 1
Enter 1 array elements:
-7
Maximum subarray sum: -7
```

Test Case 8

Input: size = 5; array = -2 -3 -1 -4 -6. Expected: Maximum subarray sum: -1. (All negative with different maximum)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 5
Enter 5 array elements:
-2 -3 -1 -4 -6
Maximum subarray sum: -1
```

Test Case 9

Input: size = 5; array = 2 -1 2 -1 2. Expected: Maximum subarray sum: 4. (Alternating signs)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 5
Enter 5 array elements:
2 -1 2 -1 2
Maximum subarray sum: 4
```

Test Case 10

Input: size = 0; array = (no elements). Expected: Invalid size message. (Invalid zero array size)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc57
Enter array size: 0
Invalid size. Enter an integer from 1 to 10000.
```